

String matching learning notes

Round 5

- MP/KMP：不浪费已匹配信息，失败只回退 pattern 指针
- BM：从右往左 + 大跳跃（bad character / good suffix）
- Trie：合并公共前缀，加速多字符串查询

本质：都是在减少“重复比较字符”。

1. Basic String Notation（基础记号）

设字符串 $A = \text{"CTCGAAAAAAGGTTTCTA"}$ 。

- substring（子串）： $A[i..j]$ ，连续片段（如 $A[3..6]$ ）。
- prefix（前缀）： $A[0..j]$ （如 "ATATA" 的前缀：A、AT、ATA...）。
- suffix（后缀）： $A[i..n-1]$ 。
- concatenation（拼接）： AB 表示 A 后接 B。

2. String Matching（匹配问题定义）

给 pattern P （长度 m ）和 text T （长度 n ），找所有 k 使得 $T[k..k+m-1] = P$ 。

例： $T = \text{"CGATACCATAGATATAGT"}$ ， $P = \text{"ATATA"}$ ，在位置 11 出现（从 0 开始数）。

3. Brute Force（暴力匹配）

思路：对每个起点 k ，把 P 从左到右与 $T[k..]$ 逐字符比；失败就 $k++$ 再试。

最坏复杂度 $O(mn)$ ：因为 n 个起点里，每个起点可能比较到 m 才失败。

4. MP/KMP（核心块）

K = Donald **Knuth**

M = James H. **Morris**

P = Vaughan **Pratt**

4.1 Morris–Pratt (MP)：思想/动机（KMP 前身）

暴力的问题：一旦在末尾失败（如 ATATA vs ATATG），明明“前面一大段已匹配”却从头开始，重复比较。

MP 的核心：利用已经匹配的信息，不回退 text。

4.2 fMP/LPS：KMP 的“前缀表”（预处理）

说明：不同教材叫法不同，但本质是同一个数组。

- **fMP**：MP/KMP 的 failure / 前缀表（记录“最长 border 的长度”）
- **LPS**：Longest Proper Prefix which is also Suffix（最长真前缀=后缀）

对 pattern 的每个位置 j ($0..m-1$)，定义 **fMP(j)**：

$P[0..j]$ 的“最长 proper suffix（不等于整个串）”，且同时也是 prefix 的长度（也叫 longest border）。

它告诉你：当在 $j+1$ 处失配时，pattern 应该跳到哪里继续。

4.3 前缀表例子：P = "ATATA"

看 $j=0$ （子串 "A"）：fMP[0]=0

看 $j=1$ （子串 "AT"）：fMP[1]=0

看 $j=2$ （子串 "ATA"）：前后缀最长是 A，长度 1 → fMP[2]=1

看 $j=3$ （子串 "ATAT"）：前后缀最长是 AT，长度 2 → fMP[3]=2

看 $j=4$ （子串 "ATATA"）：共同最长是 "ATA"，长度 3，所以 fMP(4)=3。

- 所有 proper suffix：A、TA、ATA、TATA
- 所有 proper prefix：A、AT、ATA、ATAT

该 pattern 的 fMP 数组为：[0,0,1,2,3]。

"anas" 的 fMP 数组为：[0,0,1,2,3,0]。

"TAATAAT" 的 fMP 数组为：[0,0,0,1,2,3,4]。

4.4 MP 匹配/跳转过程（search）

当已经匹配到末尾但失配时（ATATA vs ATATG），不用让 text 指针 i 回退；

只把 pattern 指针 j 改成 $j = \text{fMP}[j-1]$ ，相当于把 pattern 滑到“最长可复用的前缀位置”继续比。

4.5 MP 最重要性质与复杂度

关键性质：text 指针 i 永远不后退，

所以匹配阶段总体 $O(n)$ ；加上预处理 fMP 的 $O(m)$ ，总计 $O(n+m)$ 。

4.6 MP 伪代码要点

- 匹配成功： $i++$, $j++$ 。
- 匹配失败：
 - 若 $j > 0$ ： $j = \text{fMP}[j-1]$ (pattern 不重来，利用前缀信息)
 - 否则： $i++$ (pattern 在开头就失败，只能 text 右移)

```
P = T A A T A A T
    0 1 2 3 4 5 6
```

```
T = T A A T A A A T A T T A T A A T A A T T
    0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9
```

```
fMP = [0,0,0,1,2,3,4]
```

```
1-7   T[0..6] 对 P[0..6]
```

```
8     T[6]   vs P[3]
```

```
9     T[6]   vs P[0]
```

```
10-12 T[7..9] 对 P[0..2]
```

```
13    T[9]   vs P[0]
```

```
14    T[10]  vs P[1]
```

```
15    T[10]  vs P[0]
```

```
16-17 T[11..12] 对 P[1..2]
```

```
18    T[12]  vs P[0]
```

```
19-24 T[13..18] 对 P[1..6]
```

4.7 KMP 到底“比 MP 多了什么”

两者大 O 都是 $O(n+m)$ ，差别主要在常数和实现细节。

- MP 的跳转：失配就让 j 回到某个 border ($j = \text{fMP}[j-1]$)，继续比较。

- KMP 常见实现会把前缀表写成更方便的形式（有的叫 `next[]`），让某些情况下的“必然再次失配”能少做几次跳转/比较，因此实践中更快。

5. Boyer-Moore (BM)

BM 与 MP/KMP 的主要区别：BM 通常 pattern 从右往左 比较；

一旦失配，往往能跳更远（甚至跳过整个 pattern），因此实际经常更快。

KMP：利用已经匹配的 prefix

BM：利用已经匹配的 suffix

$fBM(j)=k$: 若 suffix $P[j..m-1]$ 已匹配成功后发生失配，pattern 至少应右移 k 位 ($m = \text{pattern length}$)

```

      0 1 2 3 4 5
P = b a n a n a
fBM=[6,6,6,2,2,2,1]
```

5.1 BM 两个核心规则

- Good Suffix Rule：已匹配的后缀尽量复用，平移到下一处可能匹配的位置。
- Bad Character Rule：看失配字符在 pattern 中“最后一次出现”的位置来决定跳多少；若失配字符根本不在 pattern 中，可直接跳过整个 pattern（例：ATATA vs C）。

5.2 BM 复杂度

最坏仍可到 $O(mn)$ ，但平均/实际表现通常很好（因为跳得多）。

6. Trie（字典树）与多字符串

reTRIEva 检索

6.1 Multiple String Comparison（多字符串查询动机）

例：一组词 Olga/Oliver/Olivia/Adam/Ada，输入 "Oli" 快速找出 Oliver、Olivia (Autocomplete)。

6.2 Trie（字典树）

Trie 把多字符串“共享前缀”合并。例：Olga、Oliver、Olivia 共享 "Ol"：root → O → l 后再分叉。

查找复杂度：沿字符路径走， $O(m)$ (m 为查询串长度)，与字符串数量 k 无关。

6.3 Trie 的三种存储方式（优缺点）

- 数组：每节点 $u.out[c]$ (如 ASCII 256)。查边 $O(1)$ ，但空间浪费。
- 排序数组：省空间；查边用二分 $O(\log b)$ (b 为该节点分支数)。
- Hash：平均快，但有额外空间/常数开销。

Round 6

“字符串匹配 = 状态转移”：

- KMP：状态=已匹配前缀长度
- AC：状态=Trie 节点
- DFA/NFA：状态=自动机节点
- Regex：先转 NFA 再匹配

1. Multiple String Matching（问题定义）

输入：多个 pattern $P=\{P_1..P_k\}$ ，一个 text T ；

输出：所有 pattern 在 T 中出现的位置。

例：patterns={ervin, verse, error, server}, $T="servererror"$,

希望一次找出 server、error 等。

2. 为什么朴素方法慢

- k 个 pattern 各跑一次 KMP： $O(kn)$
- 或 Trie：对 T 的每个 suffix 去 Trie 查，最坏仍很大（常写成 $O(n \cdot m_{\max})$ 量级）

3. AC 自动机（Aho–Corasick，重点块）

一句话：Trie + KMP 的失败跳转（failure link）。

思想：既然 KMP 能在失配时跳到更短的“可复用前缀”，那 Trie 节点也能在失配时跳到“当前路径字符串的最长 suffix（且在 Trie 中存在）”。

3.1 fAC（failure link）定义

对 Trie 中每个节点 u , $u.fAC=v$, 其中 $str(v)$ 是 $str(u)$ 的最长 proper suffix, 且该 suffix 在 Trie 中存在。

直观：已匹配 "server" 后失败, 不用回 root, 可跳到 "er" 对应节点继续。

3.2 AC 搜索过程要点

text 只扫描一遍：沿 Trie 走；如果某字符走不动, 就沿 failure link 不断跳, 直到能走或回到 root。

因此搜索复杂度 $O(n + \text{输出数量})$ ；建 Trie + 构造 failure link 的预处理通常 $O(\text{total pattern length} + \text{节点数} \cdot r)$ (取决于实现)。

3.3 failure link 如何构造 (precomputeFAC)

用 BFS 分层构建：因为父节点的 failure 已知, 子节点的 failure 才好算。

对应关系：KMP 里 $j=fMP[j-1]$ ；AC 里沿 $v=v.fAC$ 不断找更短 suffix。

4. Finite State Automata (自动机)

自动机本质是“状态图”：states + transitions + accepting states。

读入字符串时, 每读一个字符做一次状态转移。

4.1 DFA (确定有限自动机)

特点：对“某状态 + 某字符”只有一条转移, 因此当前状态唯一；

处理长度为 n 的输入是 $O(n)$ 。

4.2 NFA (非确定自动机)

特点：同一字符可有多条边；还允许 **ϵ -transition** (epsilon transition, 空跳转)。

ϵ -transition 的含义：

- **不消耗输入字符** (不读 text 的下一个字符也能走一步)
- 用来把自动机“结构拼起来” (Regex $\rightarrow$ NFA 构造里非常常见：union/concateration/star 都靠 ϵ 边连接)
- ϵ 代表 **空串** (empty string)

因此 NFA 在某一时刻可能处于多个 active states (因为可以沿 ϵ 边扩展出一批可达状态)。

NFA 可同时保持多个 active 状态。

结构常更小, 但直接模拟可能更慢 (常见上界 $O(mn)$)。

4.3 DFA vs NFA

DFA	NFA
唯一状态	多个 active 状态
快 ($O(n)$)	灵活 (但模拟可能更慢)
状态可能很大 (状态爆炸)	结构通常更小

5. Regex (正则表达式)

Regex 描述“字符串集合”。三大操作：

- concatenation : E_1E_2
- union : $E_1|E_2$
- Kleene star : E^* (重复任意次, 包含 ϵ)

优先级 (常用) : $\text{star} > \text{concatenation} > |$ 。例 : $cd|ab = (cd) / (a(b))$ 。

5.1 Regex 与 NFA / 匹配复杂度

任意 Regex 都可转成 NFA (concatenation 连接、union 加 ϵ 分叉、star 加 ϵ 回边)。

若 regex 长度 m 、text 长度 n , NFA 大小 $O(m)$, 直接模拟匹配常写 $O(mn)$ 。